

C++ Code

```
1  #include <algorithm>
2  #include <climits>
3  #include <cstdlib>
4  #include <ctime>
5  #include <functional>
6  #include <iomanip>
7  #include <iostream>
8  #include <queue>
9  #include <set>
10 #include <string>
11 #include <vector>
12 #include <map>
13 #include <numeric>
14
15 using namespace std;
16
17 // =====
18 //  GLOBAL CONSTANTS & DATA STRUCTURES
19 // =====
20
21 const int NUM_CITIES = 500;
22 const int NUM_REGIONS = 25;
23 const int MIN_WEIGHT = 5;
24 const int MAX_WEIGHT = 50;
25 const int MIN_CONN = 1;
26 const int MAX_CONN = 2;
27 const int MIN_BRIDGES = 3;
28 const int MAX_BRIDGES = 8;
29
30 vector<vector<pair<int, int>>> adjList(NUM_CITIES);
31 int totalEdges = 0;
32
33 // =====
34 //  UTILITY FUNCTIONS (Reused from Assignment 2)
35 // =====
36
37 int randomWeight() {
38     return MIN_WEIGHT + rand() % (MAX_WEIGHT - MIN_WEIGHT + 1);
39 }
40
41 void addEdge(int u, int v, int w) {
42     adjList[u].push_back({v, w});
43     adjList[v].push_back({u, w});
44     totalEdges++;
45 }
46
47 void printSeparator(char ch = '=', int len = 60) {
48     cout << " " << string(len, ch) << endl;
49 }
50
```

```
51 void printHeader(const string &title) {
52     cout << endl;
53     printSeparator('=', 65);
54     cout << "      " << title << endl;
55     printSeparator('=', 65);
56     cout << endl;
57 }
58
59 // =====
60 //  BFS & COMPONENT DETECTION
61 // =====
62
63 void bfs(int start, vector<bool> &visited, vector<int> &component) {
64     queue<int> q;
65     q.push(start);
66     visited[start] = true;
67
68     while (!q.empty()) {
69         int curr = q.front();
70         q.pop();
71         component.push_back(curr);
72
73         for (int i = 0; i < (int)adjList[curr].size(); i++) {
74             int neighbor = adjList[curr][i].first;
75             if (!visited[neighbor]) {
76                 visited[neighbor] = true;
77                 q.push(neighbor);
78             }
79         }
80     }
81 }
82
83 vector<vector<int>> findAllComponents() {
84     vector<vector<int>> components;
85     vector<bool> visited(NUM_CITIES, false);
86
87     for (int i = 0; i < NUM_CITIES; i++) {
88         if (!visited[i]) {
89             vector<int> component;
90             bfs(i, visited, component);
91             components.push_back(component);
92         }
93     }
94
95     return components;
96 }
97
98 // BFS that respects disabled nodes (for Task 3 & 5)
99 void bfsWithDisabled(int start, vector<bool> &visited,
100                     vector<int> &component,
101                     const vector<bool> &disabledNodes) {
102     queue<int> q;
103     q.push(start);
104     visited[start] = true;
```

```
105
106 while (!q.empty()) {
107     int curr = q.front();
108     q.pop();
109     component.push_back(curr);
110
111     for (int i = 0; i < (int)adjList[curr].size(); i++) {
112         int neighbor = adjList[curr][i].first;
113         if (!visited[neighbor] && !disabledNodes[neighbor]) {
114             visited[neighbor] = true;
115             q.push(neighbor);
116         }
117     }
118 }
119 }
120
121 // BFS that respects disabled nodes AND disabled edges (for Task 5)
122 void bfsWithDisabledEdges(int start, vector<bool> &visited,
123                           vector<int> &component,
124                           const vector<bool> &disabledNodes,
125                           const set<pair<int, int>> &disabledEdges) {
126     queue<int> q;
127     q.push(start);
128     visited[start] = true;
129
130     while (!q.empty()) {
131         int curr = q.front();
132         q.pop();
133         component.push_back(curr);
134
135         for (int i = 0; i < (int)adjList[curr].size(); i++) {
136             int neighbor = adjList[curr][i].first;
137             if (!visited[neighbor] && !disabledNodes[neighbor]) {
138                 int a = min(curr, neighbor);
139                 int b = max(curr, neighbor);
140                 if (disabledEdges.count({a, b}) == 0) {
141                     visited[neighbor] = true;
142                     q.push(neighbor);
143                 }
144             }
145         }
146     }
147 }
148
149 // Count components with disabled nodes
150 int countComponentsWithDisabledNodes(const vector<bool> &disabledNodes) {
151     vector<bool> visited(NUM_CITIES, false);
152     int count = 0;
153
154     for (int i = 0; i < NUM_CITIES; i++) {
155         if (!visited[i] && !disabledNodes[i]) {
156             vector<int> component;
157             bfsWithDisabled(i, visited, component, disabledNodes);
158             count++;
```

```
159     }
160 }
161
162 return count;
163 }
164
165 // Count components with disabled edges
166 int countComponentsWithDisabledEdges(const set<pair<int, int>> &disabledEdges) {
167     vector<bool> visited(NUM_CITIES, false);
168     vector<bool> disabledNodes(NUM_CITIES, false);
169     int count = 0;
170
171     for (int i = 0; i < NUM_CITIES; i++) {
172         if (!visited[i]) {
173             vector<int> component;
174             bfsWithDisabledEdges(i, visited, component, disabledNodes,
175                                 disabledEdges);
176             count++;
177         }
178     }
179
180     return count;
181 }
182
183 // =====
184 // NETWORK GENERATION
185 // =====
186
187 void generateNetwork() {
188     vector<int> region(NUM_CITIES);
189     for (int i = 0; i < NUM_CITIES; i++) {
190         region[i] = rand() % NUM_REGIONS;
191     }
192
193     vector<vector<int>> regionMembers(NUM_REGIONS);
194     for (int i = 0; i < NUM_CITIES; i++) {
195         regionMembers[region[i]].push_back(i);
196     }
197
198     cout << " Geographical Region Distribution:" << endl;
199     printSeparator('-', 45);
200
201     for (int r = 0; r < NUM_REGIONS; r++) {
202         int sz = regionMembers[r].size();
203         cout << "    Region " << setw(2) << r << ": " << setw(3) << sz
204              << " cities ";
205         int barLen = sz / 2;
206         for (int b = 0; b < barLen; b++)
207             cout << "#";
208         cout << endl;
209     }
210     cout << endl;
211
212     set<pair<int, int>> edgeSet;
```

```
213
214 for (int r = 0; r < NUM_REGIONS; r++) {
215     vector<int> &members = regionMembers[r];
216     int n = members.size();
217     if (n <= 1)
218         continue;
219
220     for (int idx = 0; idx < n; idx++) {
221         int city = members[idx];
222         int numConnections = MIN_CONN + rand() % (MAX_CONN - MIN_CONN + 1);
223
224         for (int c = 0; c < numConnections; c++) {
225             int neighborIdx = rand() % n;
226             int neighbor = members[neighborIdx];
227             if (neighbor == city)
228                 continue;
229
230             int u = min(city, neighbor);
231             int v = max(city, neighbor);
232             if (edgeSet.count({u, v}) == 0) {
233                 addEdge(u, v, randomWeight());
234                 edgeSet.insert({u, v});
235             }
236         }
237     }
238 }
239
240 int bridgeTarget = MIN_BRIDGES + rand() % (MAX_BRIDGES - MIN_BRIDGES + 1);
241 int bridgesAdded = 0;
242 int maxAttempts = 300;
243
244 cout << " Adding inter-region bridge roads..." << endl;
245
246 for (int attempt = 0; attempt < maxAttempts && bridgesAdded < bridgeTarget;
247     attempt++) {
248     int u = rand() % NUM_CITIES;
249     int v = rand() % NUM_CITIES;
250
251     if (u != v && region[u] != region[v]) {
252         int a = min(u, v);
253         int b = max(u, v);
254         if (edgeSet.count({a, b}) == 0) {
255             addEdge(a, b, randomWeight());
256             edgeSet.insert({a, b});
257             bridgesAdded++;
258         }
259     }
260 }
261
262 cout << " Bridge roads added: " << bridgesAdded << endl;
263 cout << endl;
264
265 vector<vector<int>> components = findAllComponents();
266
```

```

267     if ((int)components.size() <= 1) {
268         cout << "    [!] WARNING: Only 1 component detected." << endl;
269         cout << "        The graph is too connected." << endl;
270     } else {
271         cout << "    [OK] Network has " << components.size()
272             << " distinct clusters – realistic topology achieved!" << endl;
273     }
274 }
275
276 void printNetworkStats() {
277     double avgDegree = (2.0 * totalEdges) / NUM_CITIES;
278
279     cout << endl;
280     cout << "    Network Statistics:" << endl;
281     printSeparator('-', 45);
282     cout << "        Total Cities (Nodes) : " << NUM_CITIES << endl;
283     cout << "        Total Roads  (Edges) : " << totalEdges << endl;
284     cout << "        Average Degree      : " << fixed << setprecision(2) << avgDegree
285         << endl;
286
287     int maxDeg = 0, minDeg = NUM_CITIES;
288     int zeroDegCount = 0;
289     vector<int> degrees(NUM_CITIES);
290
291     for (int i = 0; i < NUM_CITIES; i++) {
292         degrees[i] = adjList[i].size();
293         maxDeg = max(maxDeg, degrees[i]);
294         minDeg = min(minDeg, degrees[i]);
295         if (degrees[i] == 0)
296             zeroDegCount++;
297     }
298
299     cout << "        Maximum Degree      : " << maxDeg << endl;
300     cout << "        Minimum Degree      : " << minDeg << endl;
301     cout << "        Isolated Cities     : " << zeroDegCount << endl;
302     cout << endl;
303
304     cout << "    Degree Distribution:" << endl;
305     printSeparator('-', 45);
306
307     vector<int> degFreq(maxDeg + 1, 0);
308     for (int i = 0; i < NUM_CITIES; i++) {
309         degFreq[degrees[i]]++;
310     }
311
312     for (int d = 0; d <= min(maxDeg, 12); d++) {
313         if (degFreq[d] > 0) {
314             cout << "        Degree " << setw(2) << d << ": " << setw(4) << degFreq[d]
315                 << " cities ";
316             int bar = degFreq[d] / 3;
317             for (int b = 0; b < bar; b++)
318                 cout << "|";
319             cout << endl;
320         }

```

```

321     }
322     if (maxDeg > 12) {
323         int remaining = 0;
324         for (int d = 13; d <= maxDeg; d++)
325             remaining += degFreq[d];
326         if (remaining > 0)
327             cout << "    Degree 13+: " << setw(3) << remaining << " cities" << endl;
328     }
329     cout << endl;
330
331     cout << " Sample Adjacency List (first 15 connected cities):" << endl;
332     printSeparator('-', 55);
333
334     int printed = 0;
335     for (int i = 0; i < NUM_CITIES && printed < 15; i++) {
336         if (!adjList[i].empty()) {
337             cout << "    City " << setw(3) << i << " -> ";
338             int count = 0;
339             for (int j = 0; j < (int)adjList[i].size(); j++) {
340                 if (count > 0)
341                     cout << ", ";
342                 cout << adjList[i][j].first << "(w=" << adjList[i][j].second << ")";
343                 count++;
344                 if (count >= 6 && (int)adjList[i].size() > 6) {
345                     cout << " ... [" << (adjList[i].size() - 6) << " more]";
346                     break;
347                 }
348             }
349             cout << endl;
350             printed++;
351         }
352     }
353 }
354
355 // =====
356 // UNION-FIND (Disjoint Set Union) for Kruskal's MST
357 // =====
358
359 struct UnionFind {
360     vector<int> parent, rank_;
361
362     UnionFind(int n) : parent(n), rank_(n, 0) {
363         for (int i = 0; i < n; i++)
364             parent[i] = i;
365     }
366
367     int find(int x) {
368         if (parent[x] != x)
369             parent[x] = find(parent[x]); // Path compression
370         return parent[x];
371     }
372
373     bool unite(int x, int y) {
374         int px = find(x), py = find(y);

```

```

375     if (px == py)
376         return false; // Already in same set
377     // Union by rank
378     if (rank_[px] < rank_[py])
379         swap(px, py);
380     parent[py] = px;
381     if (rank_[px] == rank_[py])
382         rank_[px]++;
383     return true;
384 }
385 };
386
387 // =====
388 // DIJKSTRA WITH DISABLED NODES & EDGES (for Tasks 3 & 4)
389 // =====
390
391 struct DijkstraResult {
392     vector<int> dist;
393     vector<int> parent;
394     bool pathExists;
395     int pathDist;
396     vector<int> path;
397 };
398
399 DijkstraResult dijkstraModified(int source, int destination,
400                                const vector<bool> &disabledNodes,
401                                const set<pair<int, int>> &disabledEdges,
402                                const vector<double> &edgeMultipliers = {}) {
403     DijkstraResult result;
404     result.dist.assign(NUM_CITIES, INT_MAX);
405     result.parent.assign(NUM_CITIES, -1);
406     result.pathExists = false;
407     result.pathDist = INT_MAX;
408
409     vector<bool> processed(NUM_CITIES, false);
410     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
411
412     result.dist[source] = 0;
413     pq.push({0, source});
414
415     while (!pq.empty()) {
416         int u = pq.top().second;
417         pq.pop();
418
419         if (processed[u])
420             continue;
421         processed[u] = true;
422
423         for (int i = 0; i < (int)adjList[u].size(); i++) {
424             int v = adjList[u][i].first;
425             int w = adjList[u][i].second;
426
427             // Skip disabled nodes
428             if (disabledNodes[v])

```



```

429         continue;
430
431         // Skip disabled edges
432         int a = min(u, v);
433         int b = max(u, v);
434         if (disabledEdges.count({a, b}) > 0)
435             continue;
436
437         // Apply traffic multiplier if provided
438         int adjustedW = w;
439         if (!edgeMultipliers.empty()) {
440             // Find edge index for multiplier
441             // We use a simple hash: use the sorted edge pair to look up
442             adjustedW = (int)(w * edgeMultipliers[a * NUM_CITIES + b]);
443         }
444
445         if (result.dist[u] != INT_MAX && result.dist[u] + adjustedW < result.dist[v]) {
446             result.dist[v] = result.dist[u] + adjustedW;
447             result.parent[v] = u;
448             pq.push({result.dist[v], v});
449         }
450     }
451 }
452
453 // Reconstruct path to destination
454 if (destination >= 0 && destination < NUM_CITIES &&
455     result.dist[destination] != INT_MAX) {
456     result.pathExists = true;
457     result.pathDist = result.dist[destination];
458
459     int curr = destination;
460     while (curr != -1) {
461         result.path.push_back(curr);
462         curr = result.parent[curr];
463     }
464     reverse(result.path.begin(), result.path.end());
465 }
466
467 return result;
468 }
469
470 // Simpler Dijkstra with traffic multiplier map (for Task 4)
471 DijkstraResult dijkstraWithTraffic(int source, int destination,
472                                     const map<pair<int,int>, double> &trafficMult) {
473     DijkstraResult result;
474     result.dist.assign(NUM_CITIES, INT_MAX);
475     result.parent.assign(NUM_CITIES, -1);
476     result.pathExists = false;
477     result.pathDist = INT_MAX;
478
479     vector<bool> processed(NUM_CITIES, false);
480     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;
481
482     result.dist[source] = 0;

```

```
483     pq.push({0, source});
484
485     while (!pq.empty()) {
486         int u = pq.top().second;
487         pq.pop();
488
489         if (processed[u])
490             continue;
491         processed[u] = true;
492
493         for (int i = 0; i < (int)adjList[u].size(); i++) {
494             int v = adjList[u][i].first;
495             int w = adjList[u][i].second;
496
497             int a = min(u, v);
498             int b = max(u, v);
499
500             double mult = 1.0;
501             auto it = trafficMult.find({a, b});
502             if (it != trafficMult.end()) {
503                 mult = it->second;
504             }
505
506             int adjustedW = (int)(w * mult);
507
508             if (result.dist[u] != INT_MAX && result.dist[u] + adjustedW < result.dist[v]) {
509                 result.dist[v] = result.dist[u] + adjustedW;
510                 result.parent[v] = u;
511                 pq.push({result.dist[v], v});
512             }
513         }
514     }
515
516     if (destination >= 0 && destination < NUM_CITIES &&
517         result.dist[destination] != INT_MAX) {
518         result.pathExists = true;
519         result.pathDist = result.dist[destination];
520
521         int curr = destination;
522         while (curr != -1) {
523             result.path.push_back(curr);
524             curr = result.parent[curr];
525         }
526         reverse(result.path.begin(), result.path.end());
527     }
528
529     return result;
530 }
531
532 // =====
533 // TASK 1: MST OPTIMIZATION (Kruskal's Algorithm)
534 // =====
535
536 void task1_MSTOptimization() {
```

```
537     printHeader("TASK 1: MST OPTIMIZATION");
538
539     cout << "   Minimum Spanning Tree Execution" << endl;
540     cout << "   Constructs a cost-minimized network connecting all reachable" << endl;
541     cout << "   nodes with absolute efficiency." << endl;
542     cout << endl;
543     cout << "   Algorithmic Method: Kruskal's Algorithm with Union-Find" << endl;
544     printSeparator('-', 55);
545     cout << endl;
546
547     // Step 1: Collect all edges
548     struct Edge {
549         int u, v, w;
550     };
551     vector<Edge> edges;
552     set<pair<int, int>> seen;
553
554     long long originalCost = 0;
555
556     for (int u = 0; u < NUM_CITIES; u++) {
557         for (int i = 0; i < (int)adjList[u].size(); i++) {
558             int v = adjList[u][i].first;
559             int w = adjList[u][i].second;
560             int a = min(u, v);
561             int b = max(u, v);
562             if (seen.count({a, b}) == 0) {
563                 edges.push_back({a, b, w});
564                 seen.insert({a, b});
565                 originalCost += w;
566             }
567         }
568     }
569
570     cout << "   Total edges collected: " << edges.size() << endl;
571     cout << endl;
572
573     // Step 2: Sort edges by weight
574     sort(edges.begin(), edges.end(),
575          [](const Edge &a, const Edge &b) { return a.w < b.w; });
576
577     // Step 3: Kruskal's algorithm with Union-Find
578     UnionFind uf(NUM_CITIES);
579     vector<Edge> mstEdges;
580     long long mstCost = 0;
581
582     for (int i = 0; i < (int)edges.size(); i++) {
583         if (uf.unite(edges[i].u, edges[i].v)) {
584             mstEdges.push_back(edges[i]);
585             mstCost += edges[i].w;
586         }
587     }
588
589     // Step 4: Display results
590     double costSaved = 0.0;
```

```

591     if (originalCost > 0)
592         costSaved = ((double)(originalCost - mstCost) / originalCost) * 100.0;
593
594     cout << "    Results:" << endl;
595     printSeparator('-', 55);
596     cout << "        Original Cost = " << originalCost << endl;
597     cout << "        MST Cost      = " << mstCost << endl;
598     cout << "        Cost Saved    = " << fixed << setprecision(1) << costSaved << " %"
599         << endl;
600     cout << endl;
601
602     // Count how many components the MST spans
603     // (If graph is disconnected, MST will be a forest)
604     int mstComponents = NUM_CITIES - (int)mstEdges.size();
605     cout << "        MST Edges Selected    : " << mstEdges.size() << endl;
606     cout << "        Spanning Components    : " << mstComponents << endl;
607     cout << endl;
608
609     // Print selected roads
610     cout << "    Selected Roads:" << endl;
611     printSeparator('-', 45);
612
613     int displayLimit = min((int)mstEdges.size(), 30);
614     for (int i = 0; i < displayLimit; i++) {
615         cout << "        (" << mstEdges[i].u << ", " << mstEdges[i].v
616             << ") weight = " << mstEdges[i].w << endl;
617     }
618
619     if ((int)mstEdges.size() > displayLimit) {
620         cout << "        ... [+] << (mstEdges.size() - displayLimit)
621             << " more edges]" << endl;
622     }
623
624     cout << endl;
625     cout << "    [All " << mstEdges.size()
626         << " structural spans successfully written to path buffer]" << endl;
627 }
628
629 // =====
630 // TASK 2: STRATEGIC CITY IDENTIFICATION (Degree Centrality)
631 // =====
632
633 void task2_StrategicCityIdentification() {
634     printHeader("TASK 2: STRATEGIC CITY IDENTIFICATION");
635
636     cout << "    Connectivity Centroid Calculations" << endl;
637     cout << "    Identify the most influential nodes using centrality analysis" << endl;
638     cout << "    to map urbanization priority indicators." << endl;
639     printSeparator('-', 55);
640     cout << endl;
641
642     // Compute degree centrality for every vertex
643     struct CityDegree {
644         int cityId;

```

```
645     int degree;
646 };
647
648 vector<CityDegree> cities(NUM_CITIES);
649 for (int i = 0; i < NUM_CITIES; i++) {
650     cities[i].cityId = i;
651     cities[i].degree = adjList[i].size();
652 }
653
654 // Sort by degree descending
655 sort(cities.begin(), cities.end(),
656      [](const CityDegree &a, const CityDegree &b) {
657          return a.degree > b.degree;
658      });
659
660 // Print Top 10 rankings
661 cout << " Top 10 Cities by Degree Centrality:" << endl;
662 printSeparator('-', 55);
663
664 int topCount = min(10, NUM_CITIES);
665 for (int i = 0; i < topCount; i++) {
666     cout << " Rank " << setw(2) << (i + 1) << " -> City " << setw(3)
667         << cities[i].cityId << " -> Degree " << cities[i].degree << endl;
668 }
669
670 cout << endl;
671
672 // Strategic Module: Automated recommendations
673 cout << " Top Development Candidates:" << endl;
674 printSeparator('-', 55);
675
676 if (topCount >= 1) {
677     cout << " * Airport: City " << cities[0].cityId << endl;
678     cout << "      \"City " << cities[0].cityId << " is connected to "
679         << cities[0].degree << " other cities and is a" << endl;
680     cout << "      strong candidate for an international airport.\"\" << endl;
681     cout << endl;
682 }
683 if (topCount >= 2) {
684     cout << " * Logistics Hub: City " << cities[1].cityId << endl;
685     cout << "      \"City " << cities[1].cityId << " has " << cities[1].degree
686         << " connections making it ideal for a logistics hub.\"\" << endl;
687     cout << endl;
688 }
689 if (topCount >= 3) {
690     cout << " * Railway Junction: City " << cities[2].cityId << endl;
691     cout << "      \"City " << cities[2].cityId << " with " << cities[2].degree
692         << " connections is optimal for a railway junction.\"\" << endl;
693     cout << endl;
694 }
695
696 // Additional: Development priority zones (cities with degree >= average * 2)
697 double avgDegree = (2.0 * totalEdges) / NUM_CITIES;
698 int threshold = (int)(avgDegree * 1.5);
```

```

699
700     cout << "   Development Priority Zones (degree >= " << threshold << "):" << endl;
701     printSeparator('-', 55);
702
703     int priorityCount = 0;
704     for (int i = 0; i < NUM_CITIES; i++) {
705         if (cities[i].degree >= threshold) {
706             if (priorityCount < 20) {
707                 cout << "       City " << setw(3) << cities[i].cityId
708                     << " | Degree: " << setw(3) << cities[i].degree
709                     << " | Priority Rank: " << (i + 1) << endl;
710             }
711             priorityCount++;
712         }
713     }
714     if (priorityCount > 20) {
715         cout << "       ... [" << (priorityCount - 20) << " more priority cities]"
716             << endl;
717     }
718     if (priorityCount == 0) {
719         cout << "       No cities exceed the priority threshold." << endl;
720     }
721
722     cout << endl;
723     cout << "   Recommendations generated:" << endl;
724     if (topCount >= 1)
725         cout << "       \"City " << cities[0].cityId << " is connected to "
726             << cities[0].degree << " other cities and is a" << endl
727             << "       strong candidate for an international airport.\"\" << endl;
728 }
729
730 // =====
731 // TASK 3: DISASTER RECOVERY ROUTING
732 // =====
733
734 void task3_DisasterRecoveryRouting() {
735     printHeader("TASK 3: DISASTER RECOVERY ROUTING");
736
737     cout << "   Dynamic Network Disruption Scenarios" << endl;
738     cout << "   Evaluate alternate routes when natural disasters destroy" << endl;
739     cout << "   cities (node removal) or roads (edge removal)." << endl;
740     printSeparator('-', 55);
741     cout << endl;
742
743     // Step 1: Choose removal type
744     int removalType;
745     cout << "   Select disruption type:" << endl;
746     cout << "       [1] Remove Cities (Node Removal)" << endl;
747     cout << "       [2] Remove Roads (Edge Removal)" << endl;
748     cout << "   Enter choice (1 or 2): ";
749     cin >> removalType;
750     cout << endl;
751
752     vector<bool> disabledNodes(NUM_CITIES, false);

```

```

753     set<pair<int, int>> disabledEdges;
754
755     if (removalType == 1) {
756         // Node removal
757         int numRemove;
758         cout << "   How many cities to remove? ";
759         cin >> numRemove;
760
761         cout << "   Enter city IDs to remove (0 to " << NUM_CITIES - 1 << "):" << endl;
762         for (int i = 0; i < numRemove; i++) {
763             int city;
764             cout << "       City " << (i + 1) << ": ";
765             cin >> city;
766             if (city >= 0 && city < NUM_CITIES) {
767                 disabledNodes[city] = true;
768                 cout << "       [!] City " << city << " destroyed by disaster." << endl;
769             } else {
770                 cout << "       [ERROR] Invalid city ID. Skipping." << endl;
771             }
772         }
773     } else {
774         // Edge removal
775         int numRemove;
776         cout << "   How many roads to remove? ";
777         cin >> numRemove;
778
779         cout << "   Enter road endpoints (city1 city2):" << endl;
780         for (int i = 0; i < numRemove; i++) {
781             int u, v;
782             cout << "       Road " << (i + 1) << " (u v): ";
783             cin >> u >> v;
784             int a = min(u, v);
785             int b = max(u, v);
786             disabledEdges.insert({a, b});
787             cout << "       [!] Road (" << a << ", " << b
788                 << ") destroyed by disaster." << endl;
789         }
790     }
791
792     cout << endl;
793
794     // Step 2: Get source cities and destination
795     int srcA, srcB, dest;
796     cout << "   Enter Source City A: ";
797     cin >> srcA;
798     cout << "   Enter Source City B: ";
799     cin >> srcB;
800     cout << "   Enter Destination City D: ";
801     cin >> dest;
802     cout << endl;
803
804     // Validate inputs
805     if (srcA < 0 || srcA >= NUM_CITIES || srcB < 0 || srcB >= NUM_CITIES ||
806         dest < 0 || dest >= NUM_CITIES) {

```

```
807     cout << " [ERROR] Invalid city IDs. Aborting task." << endl;
808     return;
809 }
810
811 if (disabledNodes[srcA] || disabledNodes[srcB] || disabledNodes[dest]) {
812     cout << " [ERROR] Source or destination city has been destroyed!" << endl;
813     if (disabledNodes[srcA])
814         cout << " City " << srcA << " (Source A) is destroyed." << endl;
815     if (disabledNodes[srcB])
816         cout << " City " << srcB << " (Source B) is destroyed." << endl;
817     if (disabledNodes[dest])
818         cout << " City " << dest << " (Destination D) is destroyed." << endl;
819     return;
820 }
821
822 // Step 3: Compute shortest paths
823 printSeparator('-', 55);
824 cout << " Computing disaster recovery routes..." << endl;
825 cout << endl;
826
827 // Path A -> D
828 DijkstraResult resultA = dijkstraModified(srcA, dest, disabledNodes, disabledEdges);
829
830 cout << " Source A Path: ";
831 if (resultA.pathExists) {
832     cout << "Distance = " << resultA.pathDist << " km | Path: ";
833     for (int i = 0; i < (int)resultA.path.size(); i++) {
834         cout << resultA.path[i];
835         if (i < (int)resultA.path.size() - 1)
836             cout << " -> ";
837     }
838     cout << endl;
839 } else {
840     cout << "No valid route available after disaster." << endl;
841 }
842
843 // Path B -> D
844 DijkstraResult resultB = dijkstraModified(srcB, dest, disabledNodes, disabledEdges);
845
846 cout << " Source B Path: ";
847 if (resultB.pathExists) {
848     cout << "Distance = " << resultB.pathDist << " km | Path: ";
849     for (int i = 0; i < (int)resultB.path.size(); i++) {
850         cout << resultB.path[i];
851         if (i < (int)resultB.path.size() - 1)
852             cout << " -> ";
853     }
854     cout << endl;
855 } else {
856     cout << "No valid route available after disaster." << endl;
857 }
858
859 cout << endl;
860
```



```

861 // Fallback state
862 if (!resultA.pathExists && !resultB.pathExists) {
863     cout << " Fallback state (If path does not exist):" << endl;
864     cout << " \"No valid route available after disaster.\"\" << endl;
865 } else {
866     // Recommend the shorter path
867     if (resultA.pathExists && resultB.pathExists) {
868         if (resultA.pathDist <= resultB.pathDist) {
869             cout << " Recommended Route: Source A (shorter by "
870                 << (resultB.pathDist - resultA.pathDist) << " km)" << endl;
871         } else {
872             cout << " Recommended Route: Source B (shorter by "
873                 << (resultA.pathDist - resultB.pathDist) << " km)" << endl;
874         }
875     } else if (resultA.pathExists) {
876         cout << " Only Source A route is viable." << endl;
877     } else {
878         cout << " Only Source B route is viable." << endl;
879     }
880 }
881 }
882
883 // =====
884 // TASK 4: TRAFFIC-AWARE SMART ROUTING
885 // =====
886
887 void task4_TrafficAwareRouting() {
888     printHeader("TASK 4: TRAFFIC-AWARE SMART ROUTING");
889
890     cout << " Dynamic Edge Weights" << endl;
891     cout << " Simulates real-world traffic flows by updating the path metrics" << endl;
892     cout << " in real-time based on congestion states." << endl;
893     printSeparator('-', 55);
894     cout << endl;
895
896     // Get source and destination
897     int source, destination;
898     cout << " Enter Source City: ";
899     cin >> source;
900     cout << " Enter Destination City: ";
901     cin >> destination;
902     cout << endl;
903
904     if (source < 0 || source >= NUM_CITIES || destination < 0 ||
905         destination >= NUM_CITIES) {
906         cout << " [ERROR] Invalid city IDs. Aborting task." << endl;
907         return;
908     }
909
910     // Traffic scenario selection
911     int scenario;
912     cout << " Select Traffic Scenario:" << endl;
913     cout << " [1] Random Traffic - each road gets a random traffic state" << endl;
914     cout << " [2] Peak Hours - most roads have Medium/High traffic" << endl;

```

```
915     cout << "    [3] Low Traffic      - most roads have Low traffic" << endl;
916     cout << "  Enter choice (1-3): ";
917     cin >> scenario;
918     cout << endl;
919
920     // Build traffic multiplier map
921     // Low = 1.0, Medium = 1.5, High = 2.0
922     map<pair<int, int>, double> trafficMult;
923     map<pair<int, int>, string> trafficState;
924
925     set<pair<int, int>> allEdges;
926     for (int u = 0; u < NUM_CITIES; u++) {
927         for (int i = 0; i < (int)adjList[u].size(); i++) {
928             int v = adjList[u][i].first;
929             int a = min(u, v);
930             int b = max(u, v);
931             allEdges.insert({a, b});
932         }
933     }
934
935     int lowCount = 0, medCount = 0, highCount = 0;
936
937     for (auto &edge : allEdges) {
938         double mult;
939         string state;
940
941         if (scenario == 1) {
942             // Random
943             int r = rand() % 3;
944             if (r == 0) {
945                 mult = 1.0;
946                 state = "Low";
947                 lowCount++;
948             } else if (r == 1) {
949                 mult = 1.5;
950                 state = "Medium";
951                 medCount++;
952             } else {
953                 mult = 2.0;
954                 state = "High";
955                 highCount++;
956             }
957         } else if (scenario == 2) {
958             // Peak hours: 20% Low, 40% Medium, 40% High
959             int r = rand() % 10;
960             if (r < 2) {
961                 mult = 1.0;
962                 state = "Low";
963                 lowCount++;
964             } else if (r < 6) {
965                 mult = 1.5;
966                 state = "Medium";
967                 medCount++;
968             } else {
```

```

969         mult = 2.0;
970         state = "High";
971         highCount++;
972     }
973 } else {
974     // Low traffic: 70% Low, 20% Medium, 10% High
975     int r = rand() % 10;
976     if (r < 7) {
977         mult = 1.0;
978         state = "Low";
979         lowCount++;
980     } else if (r < 9) {
981         mult = 1.5;
982         state = "Medium";
983         medCount++;
984     } else {
985         mult = 2.0;
986         state = "High";
987         highCount++;
988     }
989 }
990
991 trafficMult[edge] = mult;
992 trafficState[edge] = state;
993 }
994
995 cout << " Traffic Distribution:" << endl;
996 printSeparator('-', 45);
997 cout << "    Low Traffic    : " << lowCount << " roads (1.0x)" << endl;
998 cout << "    Medium Traffic : " << medCount << " roads (1.5x)" << endl;
999 cout << "    High Traffic  : " << highCount << " roads (2.0x)" << endl;
1000 cout << endl;
1001
1002 // Compute normal route (no traffic)
1003 map<pair<int, int>, double> noTraffic; // Empty = all 1.0x
1004 DijkstraResult normalResult = dijkstraWithTraffic(source, destination, noTraffic);
1005
1006 // Compute traffic-aware route
1007 DijkstraResult trafficResult = dijkstraWithTraffic(source, destination,
1008 trafficMult);
1009
1010 // Display results
1011 cout << " Rerouting Output Metrics:" << endl;
1012 printSeparator('-', 55);
1013
1014 cout << " Normal Route:" << endl;
1015 if (normalResult.pathExists) {
1016     cout << "    Distance = " << normalResult.pathDist << " km" << endl;
1017     cout << "    Path: ";
1018     int displayLimit = min((int)normalResult.path.size(), 15);
1019     for (int i = 0; i < displayLimit; i++) {
1020         cout << normalResult.path[i];
1021         if (i < displayLimit - 1)

```

```

1022     }
1023     if ((int)normalResult.path.size() > displayLimit)
1024         cout << " -> ...";
1025     cout << endl;
1026 } else {
1027     cout << "    No route available between these cities." << endl;
1028 }
1029
1030 cout << endl;
1031
1032 cout << "  Traffic Route:" << endl;
1033 if (trafficResult.pathExists) {
1034     cout << "    Distance = " << trafficResult.pathDist << " km" << endl;
1035     cout << "    Path: ";
1036     int displayLimit = min((int)trafficResult.path.size(), 15);
1037     for (int i = 0; i < displayLimit; i++) {
1038         cout << trafficResult.path[i];
1039         if (i < displayLimit - 1)
1040             cout << " -> ";
1041     }
1042     if ((int)trafficResult.path.size() > displayLimit)
1043         cout << " -> ...";
1044     cout << endl;
1045
1046     if (normalResult.pathExists && normalResult.pathDist > 0) {
1047         double delay = ((double)(trafficResult.pathDist - normalResult.pathDist) /
1048             normalResult.pathDist) *
1049             100.0;
1050         cout << "    Delay = " << fixed << setprecision(1) << delay << "%" << endl;
1051     }
1052 } else {
1053     cout << "    No route available between these cities." << endl;
1054 }
1055
1056 cout << endl;
1057 cout << "  [System actively calculated congestion bypass routes" << endl;
1058 cout << "    based on elevated edge penalties]" << endl;
1059 }
1060
1061 // =====
1062 // TASK 5: CRITICAL INFRASTRUCTURE ANALYSIS
1063 // =====
1064
1065 void task5_CriticalInfrastructureAnalysis() {
1066     printHeader("TASK 5: CRITICAL INFRASTRUCTURE ANALYSIS");
1067
1068     cout << "  Systemic Vulnerability Analysis" << endl;
1069     cout << "  Diagnoses specific single points of failure that cause extreme" << endl;
1070     cout << "  structural fractures inside the system." << endl;
1071     printSeparator('-', 55);
1072     cout << endl;
1073
1074     // Get baseline component count
1075     vector<vector<int>> baseComponents = findAllComponents();

```

```
1076     int baseCount = baseComponents.size();
1077
1078     cout << "   Baseline connected components: " << baseCount << endl;
1079     cout << endl;
1080
1081     // ---- Node Elimination ----
1082     cout << "   Phase 1: Node Elimination Analysis" << endl;
1083     cout << "   Temporarily removing each city node and measuring disruption..." << endl;
1084     printSeparator('-', 55);
1085     cout << endl;
1086
1087     int mostCriticalCity = -1;
1088     int maxDisconnectedByCity = 0;
1089     int maxComponentsByCity = baseCount;
1090
1091     // For efficiency, only test nodes that have edges
1092     int testedNodes = 0;
1093     for (int city = 0; city < NUM_CITIES; city++) {
1094         if (adjList[city].empty())
1095             continue; // Isolated nodes won't affect connectivity
1096
1097         testedNodes++;
1098
1099         // Temporarily disable this city
1100         vector<bool> disabled(NUM_CITIES, false);
1101         disabled[city] = true;
1102
1103         int compCount = countComponentsWithDisabledNodes(disabled);
1104
1105         // The removed node itself doesn't count as a component
1106         // Compare with baseline minus consideration for the removed node
1107         if (compCount > maxComponentsByCity) {
1108             maxComponentsByCity = compCount;
1109             maxDisconnectedByCity = compCount;
1110             mostCriticalCity = city;
1111         }
1112     }
1113
1114     cout << "   Nodes tested: " << testedNodes << " (skipped "
1115           << (NUM_CITIES - testedNodes) << " isolated nodes)" << endl;
1116     cout << endl;
1117
1118     if (mostCriticalCity >= 0) {
1119         cout << "   Most Critical City:" << endl;
1120         cout << "       City " << mostCriticalCity << endl;
1121         cout << endl;
1122         cout << "   Disconnected Components Created:" << endl;
1123         cout << "       " << maxDisconnectedByCity << endl;
1124     } else {
1125         cout << "   No critical city found (all nodes are isolated)." << endl;
1126     }
1127
1128     cout << endl;
1129
```

```

1130 // ---- Edge Elimination ----
1131 cout << " Phase 2: Edge Elimination Analysis" << endl;
1132 cout << " Temporarily removing each road edge and measuring disruption..." << endl;
1133 printSeparator('-', 55);
1134 cout << endl;
1135
1136 // Collect all unique edges
1137 struct EdgeInfo {
1138     int u, v;
1139 };
1140 vector<EdgeInfo> allEdges;
1141 set<pair<int, int>> seen;
1142
1143 for (int u = 0; u < NUM_CITIES; u++) {
1144     for (int i = 0; i < (int)adjList[u].size(); i++) {
1145         int v = adjList[u][i].first;
1146         int a = min(u, v);
1147         int b = max(u, v);
1148         if (seen.count({a, b}) == 0) {
1149             allEdges.push_back({a, b});
1150             seen.insert({a, b});
1151         }
1152     }
1153 }
1154
1155 int mostCriticalEdgeU = -1, mostCriticalEdgeV = -1;
1156 int maxComponentsByEdge = baseCount;
1157
1158 for (int i = 0; i < (int)allEdges.size(); i++) {
1159     set<pair<int, int>> disabledEdge;
1160     disabledEdge.insert({allEdges[i].u, allEdges[i].v});
1161
1162     int compCount = countComponentsWithDisabledEdges(disabledEdge);
1163
1164     if (compCount > maxComponentsByEdge) {
1165         maxComponentsByEdge = compCount;
1166         mostCriticalEdgeU = allEdges[i].u;
1167         mostCriticalEdgeV = allEdges[i].v;
1168     }
1169 }
1170
1171 cout << " Edges tested: " << allEdges.size() << endl;
1172 cout << endl;
1173
1174 if (mostCriticalEdgeU >= 0) {
1175     cout << " Most Critical Road:" << endl;
1176     cout << " (" << mostCriticalEdgeU << ", " << mostCriticalEdgeV << ")" << endl;
1177     cout << endl;
1178     cout << " Components after removal: " << maxComponentsByEdge << endl;
1179 } else {
1180     cout << " No critical road found (removing any single edge doesn't" << endl;
1181     cout << " increase disconnected components)." << endl;
1182 }
1183

```

```

1184     cout << endl;
1185
1186     // Final diagnostic output
1187     printSeparator('-', 55);
1188     cout << " Critical Diagnostic Output:" << endl;
1189     printSeparator('-', 55);
1190
1191     if (mostCriticalCity >= 0) {
1192         cout << " [Removal of City " << mostCriticalCity
1193             << " breaks network graph into " << maxDisconnectedByCity
1194             << " disconnected subgrids]" << endl;
1195     }
1196     if (mostCriticalEdgeU >= 0) {
1197         cout << " [Removal of Road (" << mostCriticalEdgeU << ", "
1198             << mostCriticalEdgeV << ") splits network into "
1199             << maxComponentsByEdge << " components]" << endl;
1200     }
1201 }
1202
1203 // =====
1204 // COMPLEXITY ANALYSIS SUMMARY
1205 // =====
1206
1207 void printComplexityAnalysis() {
1208     printHeader("COMPLEXITY ANALYSIS SUMMARY");
1209
1210     cout << " +-----+-----+-----+
1211 +> << endl;
1212     cout << " | Algorithm | Time Complexity | Space
1213 |> << endl;
1214     cout << " +-----+-----+-----+
1215 +> << endl;
1216     cout << " | Network Generation | O(V * k) | O(V + E)
1217 |> << endl;
1218     cout << " | Kruskal's MST (Union-Find) | O(E log E) | O(V + E)
1219 |> << endl;
1220     cout << " | Degree Centrality Ranking | O(V log V) | O(V)
1221 |> << endl;
1222     cout << " | Dijkstra (Disaster Recovery) | O((V + E) log V) | O(V)
1223 |> << endl;
1224     cout << " | Dijkstra (Traffic-Aware) | O((V + E) log V) | O(V + E)
1225 |> << endl;
1226     cout << " | Node Elimination (Critical Infra) | O(V * (V + E)) | O(V)
1227 |> << endl;
1228     cout << " | Edge Elimination (Critical Infra) | O(E * (V + E)) | O(V + E)
1229 |> << endl;
1230     cout << " +-----+-----+-----+
1231 +> << endl;
1232     cout << endl;
1233     cout << " V = " << NUM_CITIES << " cities | E = " << totalEdges
1234         << " roads | k = max connections per city" << endl;
1235 }
1236
1237 // =====

```

```
1227 // MAIN
1228 // =====
1229
1230 int main() {
1231     srand(time(0));
1232
1233     // Banner
1234     cout << endl;
1235     printSeparator('*', 65);
1236     cout << "    SMART TRANSPORTATION NETWORK – FINAL ASSIGNMENT" << endl;
1237     cout << "    Graph Theory Based Infrastructure Analysis" << endl;
1238     cout << "    Cities: " << NUM_CITIES << " | Regions: " << NUM_REGIONS
1239         << " | Weights: " << MIN_WEIGHT << "-" << MAX_WEIGHT << endl;
1240     printSeparator('*', 65);
1241     cout << endl;
1242
1243     // Network generation (reused from Assignment 2)
1244     printHeader("NETWORK GENERATION");
1245
1246     cout << " Configuration:" << endl;
1247     cout << "    Total Cities          : " << NUM_CITIES << endl;
1248     cout << "    Geographical Regions   : " << NUM_REGIONS << endl;
1249     cout << "    Connections per City  : " << MIN_CONN << " to " << MAX_CONN << endl;
1250     cout << "    Inter-Region Bridges  : " << MIN_BRIDGES << " to " << MAX_BRIDGES
1251         << endl;
1252     cout << "    Edge Weight Range     : " << MIN_WEIGHT << " to " << MAX_WEIGHT
1253         << endl;
1254     cout << endl;
1255
1256     generateNetwork();
1257     printNetworkStats();
1258
1259     // Execute all 5 tasks sequentially
1260     task1_MSTOptimization();
1261     task2_StrategicCityIdentification();
1262     task3_DisasterRecoveryRouting();
1263     task4_TrafficAwareRouting();
1264     task5_CriticalInfrastructureAnalysis();
1265
1266     // Complexity analysis
1267     printComplexityAnalysis();
1268
1269     cout << endl;
1270     cout << " Program completed successfully." << endl;
1271     cout << endl;
1272
1273     return 0;
1274 }
1275
```